

Docker - Resumo de Estudos

O que é Docker?

O Docker é uma plataforma que permite criar, distribuir e executar aplicações dentro de ambientes isolados chamados **containers**.

Seu principal objetivo é garantir que uma aplicação funcione da mesma maneira em qualquer computador, sem depender das configurações do sistema operacional.

Em vez de instalar manualmente todas as bibliotecas e ferramentas necessárias, o Docker cria um ambiente completo que pode ser reproduzido em qualquer máquina que tenha o Docker instalado.

O que é um Container?

Um **container** é uma aplicação em execução dentro de um ambiente isolado.

Ele possui tudo o que a aplicação precisa para funcionar:

- Runtime
- Bibliotecas
- Dependências
- Configurações
- Arquivos da aplicação

Quando executamos:

```
docker run ...
```

o Docker cria um container e inicia a aplicação.

Podemos pensar no container como um programa aberto e funcionando.

O que é uma Imagem (Image)?

Uma **imagem** é um pacote pronto que contém tudo o que a aplicação precisa para ser executada.

Ela é criada a partir de um Dockerfile.

Uma imagem normalmente contém:

- Sistema operacional básico
- Runtime (.NET, Java, Python...)
- Bibliotecas
- Dependências
- Aplicação compilada

A imagem **não está executando**.

Ela serve como modelo para criar containers.

Uma mesma imagem pode gerar vários containers.

O que é um Dockerfile?

O **Dockerfile** é um arquivo de texto que contém todas as instruções para construir uma imagem.

Ele funciona como uma receita.

Exemplo:

- Baixe o .NET
- Copie o projeto
- Instale as bibliotecas
- Compile a aplicação
- Defina a porta
- Execute o programa

Quando executamos:

```
docker build
```

o Docker lê esse arquivo e cria uma imagem.

O que é Docker Desktop?

Docker Desktop é o programa instalado no computador.

Ele é responsável por:

- Gerenciar imagens
- Gerenciar containers
- Gerenciar redes
- Gerenciar volumes
- Executar o Docker Engine

É através dele que podemos visualizar tudo o que está acontecendo.

O que é Docker Hub?

O Docker Hub é um repositório online de imagens Docker.

É semelhante ao GitHub, mas em vez de armazenar código, ele armazena imagens.

Quando usamos:

FROM mcr.microsoft.com/dotnet/sdk:10.0

o Docker baixa essa imagem automaticamente da internet (se ela ainda não existir no computador).

O que é um Volume?

Um volume é um espaço de armazenamento permanente.

Mesmo que um container seja apagado, os dados armazenados em um volume continuam existindo.

É muito utilizado para:

- Bancos de dados
- Uploads
- Arquivos importantes

Sem volumes, os dados seriam perdidos quando o container fosse removido.

O que é uma Network?

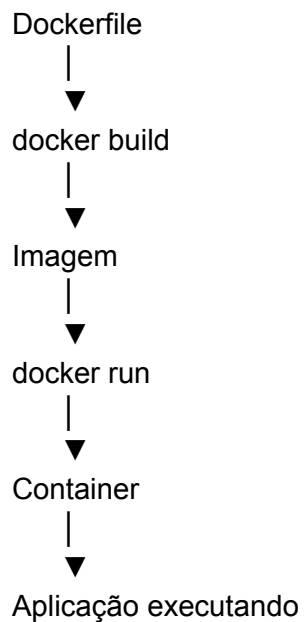
Uma network permite que vários containers se comuniquem entre si.

Exemplo:



Todos podem conversar através da mesma rede.

Fluxo de funcionamento do Docker



Explicação do Dockerfile

FROM

FROM mcr.microsoft.com/dotnet/sdk:10.0

Define qual imagem será utilizada como base.

Neste caso, utiliza a imagem oficial do .NET SDK.

WORKDIR

WORKDIR /src

Define a pasta onde os próximos comandos serão executados.

Equivale ao comando:

cd /src

COPY

COPY . .

Copia todos os arquivos do projeto para dentro do container.

RUN

RUN dotnet restore

Executa um comando durante a construção da imagem.

Neste caso:

- Baixa todas as dependências NuGet.

Outro exemplo:

RUN dotnet publish -c Release -o /app/publish

Compila a aplicação.

ENV

ENV ASPNETCORE_URLS=http://+:8080

Cria uma variável de ambiente.

Neste exemplo, informa ao ASP.NET para utilizar a porta 8080.

EXPOSE

EXPOSE 8080

Indica qual porta a aplicação utilizará.

Não abre a porta automaticamente.

A abertura acontece no comando `docker run`.

ENTRYPOINT

ENTRYPOINT ["dotnet","DockerSite.dll"]

Define o comando que será executado quando o container iniciar.

Equivale a executar:

dotnet DockerSite.dll

Comandos Docker utilizados

Verificar versão

docker --version

Mostra a versão instalada do Docker.

Verificar informações

`docker info`

Mostra informações do Docker instalado.

Testar instalação

`docker run hello-world`

Baixa a imagem "hello-world" e cria um container para verificar se o Docker está funcionando corretamente.

Construir uma imagem

`docker build -t dockersite .`

Lê o Dockerfile e cria uma imagem chamada **dockersite**.

- `-t`: define o nome da imagem.
 - `.`: indica que o Dockerfile está na pasta atual.
-

Executar uma imagem

`docker run -p 8080:8080 dockersite`

Cria um container e executa a imagem.

- `-p`: mapeia a porta do computador para a porta do container.
 - `8080:8080`: porta do computador → porta do container.
-

Listar imagens

`docker images`

Mostra todas as imagens armazenadas no computador.

Listar containers em execução

`docker ps`

Mostra os containers que estão executando.

Listar todos os containers

`docker ps -a`

Mostra todos os containers, inclusive os que estão parados.

Parar um container

`docker stop <ID_DO_CONTAINER>`

Encerra a execução de um container.

Remover um container

`docker rm <ID_DO_CONTAINER>`

Remove um container que já foi parado.

Remover uma imagem

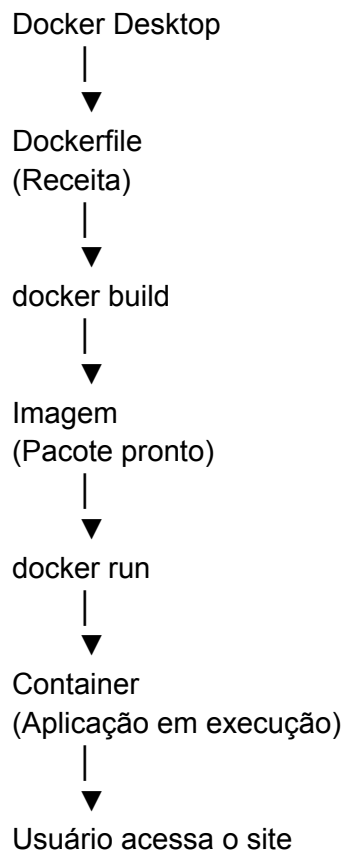
`docker rmi <NOME_DA_IMAGEM>`

Remove uma imagem armazenada no Docker.

Vantagens do Docker

- O projeto funciona da mesma forma em qualquer computador.
 - Não é necessário instalar manualmente todas as dependências da aplicação.
 - Facilita o trabalho em equipe.
 - Evita problemas de incompatibilidade de versões.
 - Isola aplicações umas das outras.
 - Facilita a implantação (deploy) em servidores e serviços de nuvem.
-

Resumo Final



Esse material cobre os principais conceitos do Docker e serve como uma boa base para revisar antes de aprender recursos mais avançados, como **Docker Compose**, **Volumes** e **Networks** em projetos com múltiplos serviços.